

Fig 1A: The path to the vulnerable file.

```php
/**
 * Class ORM
 * @package App\Paginate\Paginate\Pager
 * @property Pixie $pixie
 */
class ORM extends \PHPixie\Paginate\Pager\ORM
{
    public function __construct($pixie, $items, $page, $page_size,
$first_page_url = null)
    {
        $this->pixie = $pixie;
        $this->items =  $items;
        $this->page = $page;
        $this->page_size = $page_size;

      $filteredPageSize = $page_size instanceof VulnerableField ? $page_size-
>getFilteredValue() : $page_size;
        $filteredPage = $page instanceof VulnerableField ? $page-
>getFilteredValue() : $page;

        $this->offset = $filteredPageSize * ($filteredPage - 1);
        $this->num_items = $this->item_count();
        $this->num_pages = ceil($this->num_items / (int)$filteredPageSize);

        $this->first_page_url = $first_page_url;
    }
```

Fig 2A: Unedited code.

```php
class ORM extends \PHPixie\Paginate\Pager\ORM
{
    public function __construct($pixie, $items, $page, $page_size, $first_page_url = null)
    {
        $this->pixie = $pixie;
        $this->items =  $items;
        $this->page = $page;
        $this->page_size = $page_size;

     $filteredPageSize = (int) $filteredPageSize;
     $filteredPage = (int) $filteredPage;
```

Fig 3A: Edited code

```
GET /api/category?page=1&per_page=1000;%20select%20sleep%20(5);
HTTP/1.1
Authorization: Token ec12eebf6936b522f1d14107ac0beb1ec4a01072
User-Agent: Dalvik/2.1.0 (Linux; U; Android 9; AOSP on IA Emulator
Build/PSR1.180720.122)
Host: 192.168.50.13
Connection: keep-alive
Accept-Encoding: gzip, deflate, br
```

```
HTTP/1.1 500 Internal Server Error
Date: Tue, 05 May 2026 14:56:13 GMT
Server: Apache/2.4.18 (Ubuntu)
Content-Length: 50
Connection: close
Content-Type: application/json; charset=utf-8

{
    "message":"Division by zero",
    "code":2,
    "trace":""
}
```

Fig 4A: Vulnerability 1 fixed.



Fig 1B: Path to vulnerable file.

```php
public function action_put($data = null)
{
    if ($data === null) {
        $data = $this->request->put();
    }
    unset($data['password']);
    unset($data['photoUrl']);
    unset($data[$this->model->id_field]);

    $this->prepareData($data);
    $this->checkUpdateData($data);

    if ($this->item->photo && !$data['photo']) {
        if (is_numeric($this->item->photo)) {
            /** @var File $photo */
            $photo = $this->pixie->orm->get('file', $this->item->photo);
            if ($photo->loaded() && $photo->user_id == $this->user->id()) {
                $photo->delete();
            }
        }
    }
```

Fig 2B: Unedited code.

```php
public function action_put($data = null)
{
    if ($data === null) {
        $data = $this->request->put();
    }
    unset($data['password']);
    unset($data['photoUrl']);
    unset($data[$this->model->id_field]);

    $this->prepareData($data);
    $this->checkUpdateData($data);

    if (isset($data['first_name'])) {
if (!preg_match("/^[a-zA-Z\s\-]+$/", $data['first_name'])) {
    throw new \Exception("Error");
}
```

Fig 3B: Edited code.



Fig 4B: Vulnerability 2 fixed.



Fig 5B: Vulnerability 3 fixed.

This week we were tasked with fixing the mobile API vulnerabilities we found. As a quick recap, I found three vulnerabilities. Two SQLi and one XSS. The first vulnerability was in the per_page parameter of the shopping API. This parameter was vulnerable to SQL as I was able to input: *1000;%20select%20sleep%20(5);*. To fix this I made it so that only integers are accepted in that parameter. This vulnerability is recorded in Fig 1A – Fig 4A. The second and third vulnerabilities were in the same parameter. They were in the first_name parameter of the updating account information action. This particular parameter was not properly sanitized. By

putting, test' ', the system allows it though. This is a sign that the parameter is vulnerable to SQL. This parameter also allows an XSS attack. By inputting an XSS script into the parameter it can update the users' first name to the XSS script. That script is then run when the user logs into the website. To fix both problems at once I made it, so the first_name parameter does not accept anything but the letters A-Z, a-z. This can be seen at the bottom of Fig 3B. Fig 4B and 5B show the fixed responses.

References

Cross Site Scripting Prevention - OWASP Cheat Sheet series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

SQL Injection Prevention - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html